```
from google.colab import drive
import pandas as pd
import glob
import os
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import math
sns.set_style("whitegrid")
plt.rcParams["figure.figsize"] = (16, 20)
```

```
# Mounting My Google Drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force
```

```
df = pd.read_csv("/content/drive/MyDrive/Research Projects /Data Science and ML /Code/WQI Calculation Code/New WQ
df.head()
```

	WaterbodyName	Years	SampleDate	Alkalinity-total (as CaCO3)	Ammonia-Total (as N)	BOD - 5 days (Total)	Chloride	Conductivity @25°C	Dissolved Oxygen	ortho Phosphat (as P) unspecifie
0	ABBEYTOWN_010	2023	Feb	314.0	0.033	1.2	27.3	711.0	52.50	0.01
1	Allua	2007	Aug	14.0	0.033	1.2	10.0	71.0	61.85	0.01
2	Allua	2007	Aug	17.0	0.033	1.2	11.5	79.0	61.80	0.01
3	Allua	2007	Aug	18.0	0.033	1.2	11.3	78.0	62.40	0.01
4	Allua	2007	Sep	19.0	0.033	1.2	10.5	76.0	66.05	0.01

5 rows × 22 columns

```
df.shape
```

```
(29159, 22)
```

Initial Data Distribution Check

```
df.describe()
```

	Years	Alkalinity-total (as CaCO3)	Ammonia-Total (as N)	BOD - 5 days (Total)	Chloride	Conductivity @25°C	Dissolved Oxygen	ortho-Phosphate (as P) - unspecified	
count	29159.000000	29159.000000	29159.000000	29159.000000	29159.000000	29159.000000	29159.000000	29159.000000	29159.(
mean	2014.782537	139.858348	0.063573	1.276861	20.257858	363.039081	58.286184	0.068789	7.5
std	4.441425	95.554543	0.441630	0.559890	19.127637	182.190019	26.946092	1.010133	0.5
min	2007.000000	0.000000	0.000000	0.000000	0.000000	33.000000	0.000000	-0.004000	4.1
25%	2011.000000	55.000000	0.033000	1.200000	15.600000	229.000000	49.400000	0.017000	7.2
50%	2015.000000	127.000000	0.033000	1.200000	18.600000	356.000000	54.900000	0.019000	7.1
75%	2018.000000	214.000000	0.033000	1.200000	22.000000	501.000000	75.000000	0.022000	8.(
max	2023.000000	442.000000	40.000000	16.000000	1260.000000	4200.000000	198.000000	70.000000	9.8

Missing Value Check

```
df.isnull().sum()
```

	0
WaterbodyName	0
Years	0
SampleDate	0
Alkalinity-total (as CaCO3)	0
Ammonia-Total (as N)	0
BOD - 5 days (Total)	0
Chloride	0
Conductivity @25°C	0
Dissolved Oxygen	0
ortho-Phosphate (as P) - unspecified	0
pH	0
Temperature	0
Total Hardness (as CaCO3)	0
True Colour	0
IndWQI	0
IndWQI_Score	0
IWQI	0
IWQI_Score	0
DWQI	0
DWQI_Score	0
AWQI	0
AWQI_Score	0

dtype: int64

Dataset Description and Dictionary

```
def create_data_dictionary(df):
    data_dict = []

    for col in df.columns:
        data_type = df[col].dtype
        non_null_count = df[col].count()
        null_count = df[col].isnull().sum()
        unique_count = df[col].nunique()

        # Get example values (first 3 unique values)
        example_values = df[col].dropna().unique()[:3]

        data_dict.append({
            "Feature Name": col,
            "Data Type": str(data_type),
            "Non-Null Count": non_null_count,
            "Null Count": null_count,
            "Unique Values": unique_count,
            "Example Values": example_values
        })

    return pd.DataFrame(data_dict)

# Usage
data_dictionary = create_data_dictionary(df)

# Display
print(data_dictionary)

# Optional: Save to CSV
data_dictionary.to_csv("data_dictionary.csv", index=False)
```

```
                         Feature Name Data Type  Non-Null Count  \
0                         WaterbodyName    object           29159
1                                 Years     int64           29159
2                            SampleDate    object           29159
3           Alkalinity-total (as CaCO3)   float64           29159
4                  Ammonia-Total (as N)   float64           29159
```

```
5                   BOD – 5 days (Total)   float64           29159
6                               Chloride   float64           29159
7                     Conductivity @25°C   float64           29159
8                       Dissolved Oxygen   float64           29159
9   ortho-Phosphate (as P) – unspecified   float64           29159
10                                    pH   float64           29159
11                           Temperature   float64           29159
12             Total Hardness (as CaCO3)   float64           29159
13                           True Colour   float64           29159
14                                IndWQI    object           29159
15                          IndWQI_Score   float64           29159
16                                  IWQI    object           29159
17                           IWQI_Score   float64           29159
18                                 DWQI    object           29159
19                           DWQI_Score   float64           29159
20                                 AWQI    object           29159
21                           AWQI_Score   float64           29159

    Null Count  Unique Values  \
0            0            160
1            0             17
2            0             12
3            0            514
4            0            313
5            0            234
6            0            992
7            0            817
8            0           1807
9            0            221
10           0            364
11           0            227
12           0           1005
13           0            332
14           0              3
15           0          14494
16           0              4
17           0           2122
18           0              5
19           0          11738
20           0              4
21           0           1373

                                    Example Values
0        [ABBEYTOWN_010, Allua, ASKANAGAP STREAM_010]
1                                   [2023, 2007, 2008]
2                                      [Feb, Aug, Sep]
3                                 [314.0, 14.0, 17.0]
4                               [0.033, 0.066, 0.069]
5                                     [1.2, 1.4, 1.1]
6                                  [27.3, 10.0, 11.5]
7                                 [711.0, 71.0, 79.0]
8                                  [52.5, 61.85, 61.8]
```

```
num_features = df.select_dtypes(include=['number']).columns.tolist()
print(num_features)
```

```
['Years', 'Alkalinity-total (as CaCO3)', 'Ammonia-Total (as N)', 'BOD – 5 days (Total)', 'Chloride', 'Conductivit
```

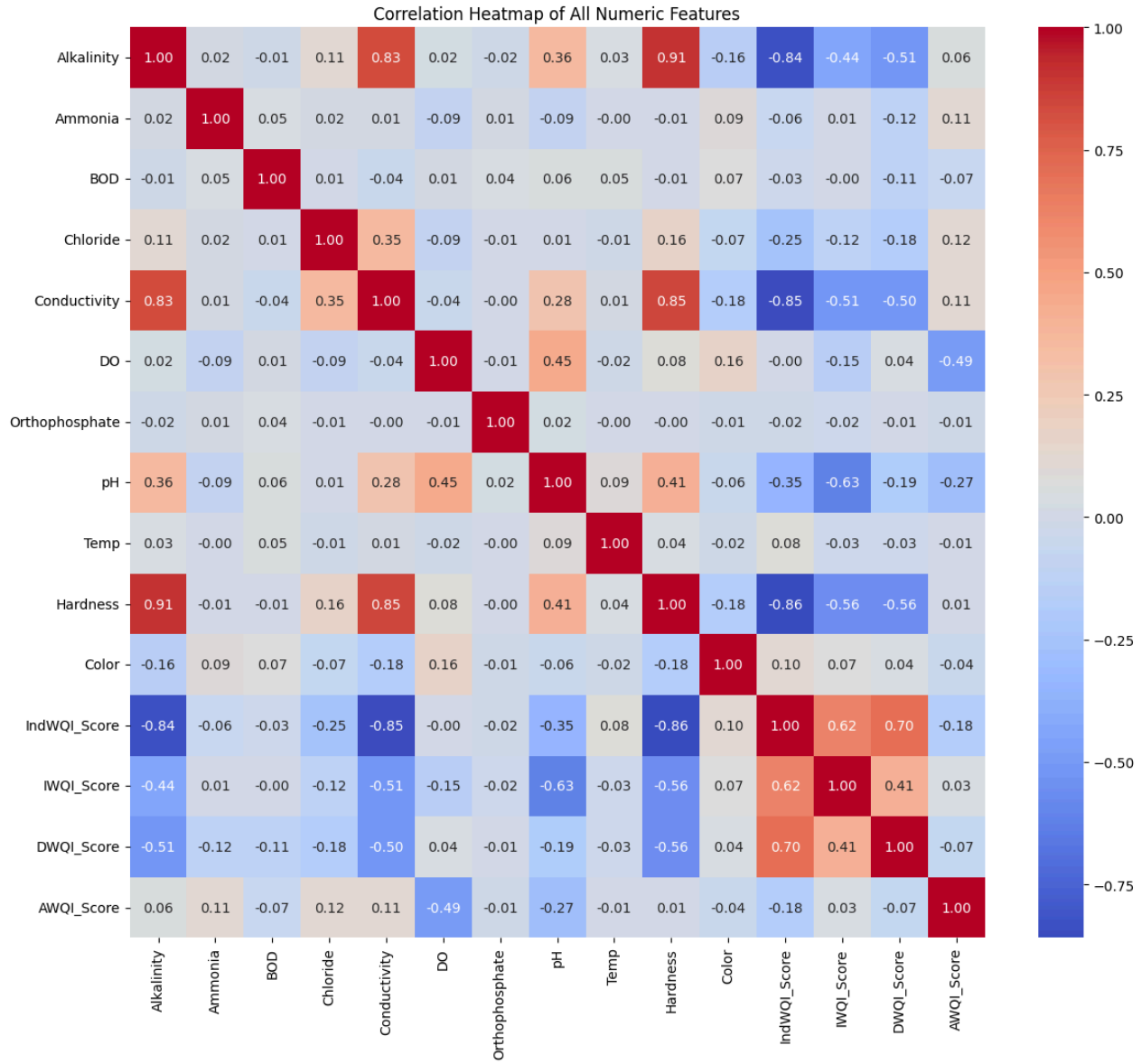
```
num_features = ['Alkalinity-total (as CaCO3)', 'Ammonia-Total (as N)', 'BOD – 5 days (Total)', 'Chloride', 'Cond
```

```
# Rename columns
df = df.rename(columns={
    "Alkalinity-total (as CaCO3)": "Alkalinity",
    "Ammonia-Total (as N)": "Ammonia",
    "BOD – 5 days (Total)": "BOD",
    "Chloride": "Chloride",
    "Conductivity @25°C": "Conductivity",
    "Dissolved Oxygen": "DO",
    "ortho-Phosphate (as P) – unspecified": "Orthophosphate",
    "pH": "pH",
    "Temperature": "Temp",
    "Total Hardness (as CaCO3)": "Hardness",
    "True Colour": "Color"
})
```

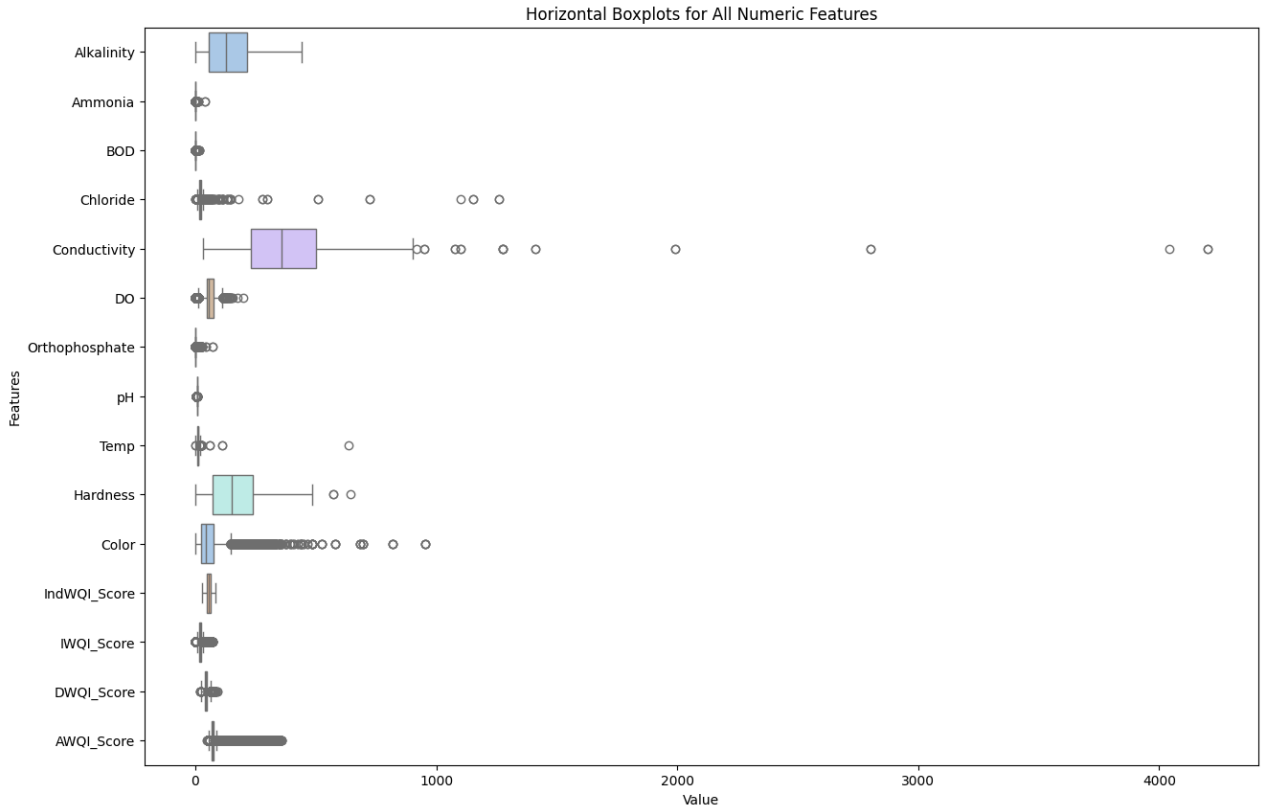
```
num_features = [
    "Alkalinity", "Ammonia", "BOD", "Chloride",
    "Conductivity", "DO", "Orthophosphate",
    "pH", "Temp", "Hardness", "Color", 'IndWQI_Score', 'IWQI_Score', 'DWQI_Score', 'AWQI_Score'
]
```

```
plt.figure(figsize=(14,12))
corr = df[num_features].corr()
sns.heatmap(corr, annot=True, fmt=".2f", cmap='coolwarm', cbar=True)
```

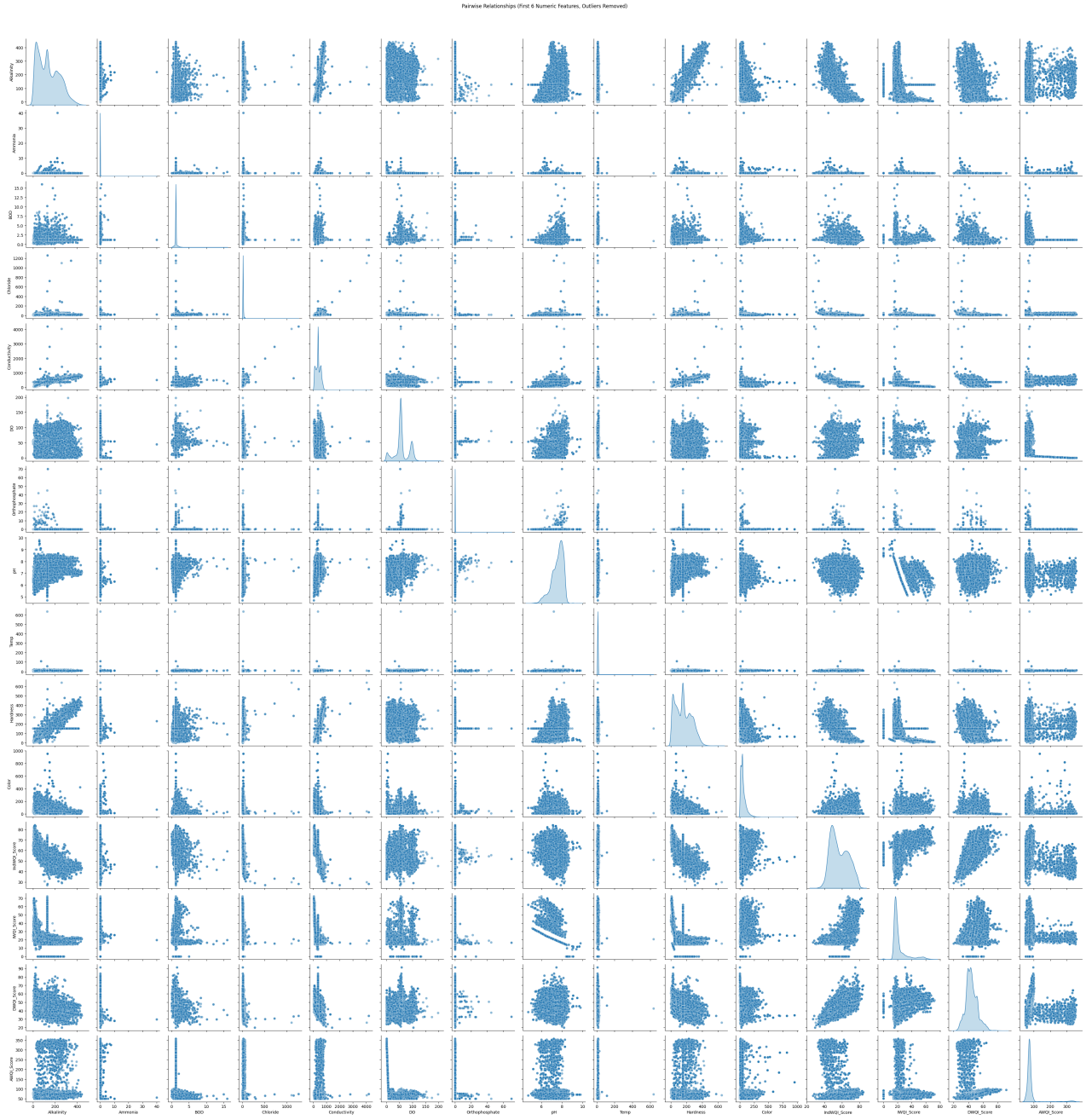
```
plt.title("Correlation Heatmap of All Numeric Features")
plt.show()
```




```
plt.figure(figsize=(15, 10))
sns.boxplot(data=df[num_features], orient="h", palette="pastel")
plt.title("Horizontal Boxplots for All Numeric Features")
plt.xlabel("Value")
plt.ylabel("Features")
plt.show()
```

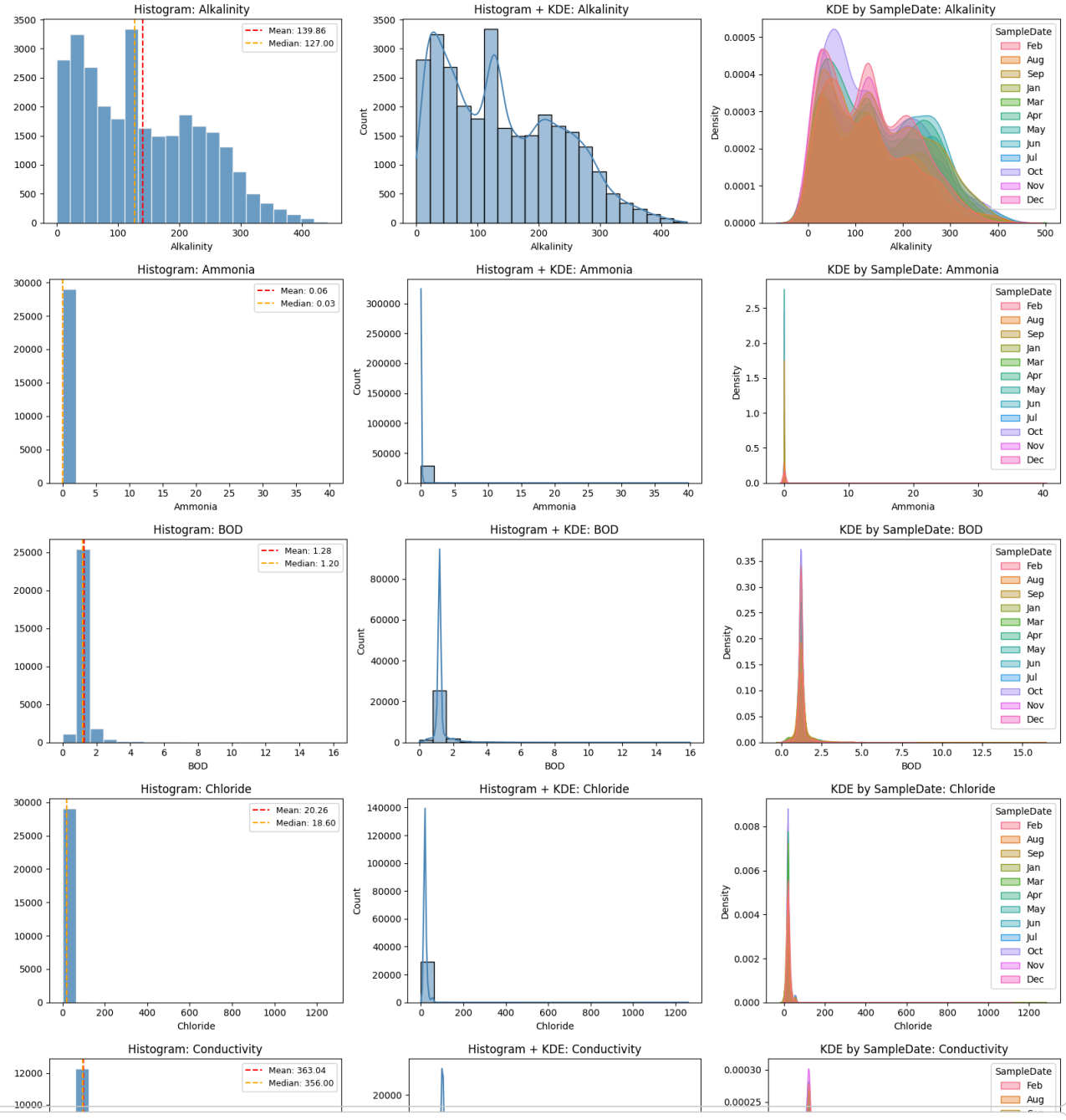



```
# Select the first 6 numeric features for initial code working check, as this code will take much execution time
# pair_features = num_features[:6]

# Pairplot with KDE on diagonals and alpha for transparency
sns.pairplot(df[num_features], diag_kind='kde', plot_kws={'alpha':0.5})
plt.suptitle("Pairwise Relationships (First 6 Numeric Features, Outliers Removed)", y=1.02)
plt.show()
```




```
# Loop over each feature
for i, feature in enumerate(num_features, 1):
    fig, axes = plt.subplots(1, 3, figsize=(16, 4))
```

```
    # 1. Basic histogram with mean/median lines
    axes[0].hist(df[feature], bins=20, color='steelblue', edgecolor='white', alpha=0.8)
    axes[0].axvline(df[feature].mean(), color='red', linestyle='--',
                    label=f"Mean: {df[feature].mean():.2f}")
    axes[0].axvline(df[feature].median(), color='orange', linestyle='--',
                    label=f"Median: {df[feature].median():.2f}")
    axes[0].set_title(f'Histogram: {feature}')
    axes[0].set_xlabel(feature)
    axes[0].legend(fontsize=9)

    # 2. Histogram + KDE
    sns.histplot(df[feature], kde=True, bins=20, ax=axes[1], color='steelblue')
    axes[1].set_title(f'Histogram + KDE: {feature}')

    # 3. KDE by categorical feature (if exists) or plain KDE
    categorical_feature = "SampleDate"  # replace if another categorical feature exists
    if categorical_feature in df.columns:
        sns.kdeplot(data=df, x=feature, hue=categorical_feature, fill=True, ax=axes[2], alpha=0.4)
        axes[2].set_title(f'KDE by {categorical_feature}: {feature}')
    else:
        sns.kdeplot(df[feature], ax=axes[2], fill=True, color='steelblue', alpha=0.4)
        axes[2].set_title(f'KDE: {feature}')

    plt.tight_layout()
    plt.show()
```




```
# Melt the dataframe so that features become a categorical column
df_melted = df[num_features].melt(var_name='Feature', value_name='Value')
```

```
# Plot
plt.figure(figsize=(15, 8))
sns.violinplot(x='Feature', y='Value', data=df_melted, palette='pastel', inner='quartile')
plt.xticks(rotation=45)
plt.title("Figure 14: Violin Plot of All Numeric Features")
plt.xlabel("Features")
plt.ylabel("Value")
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_42660/3511839748.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable t
  sns.violinplot(x='Feature', y='Value', data=df_melted, palette='pastel', inner='quartile')
```

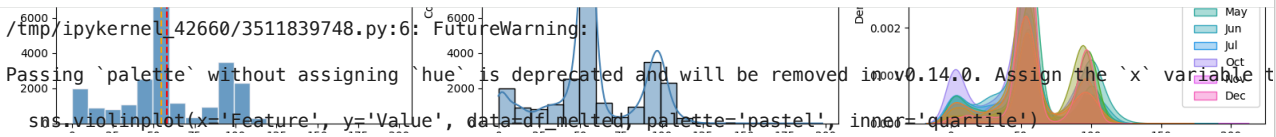
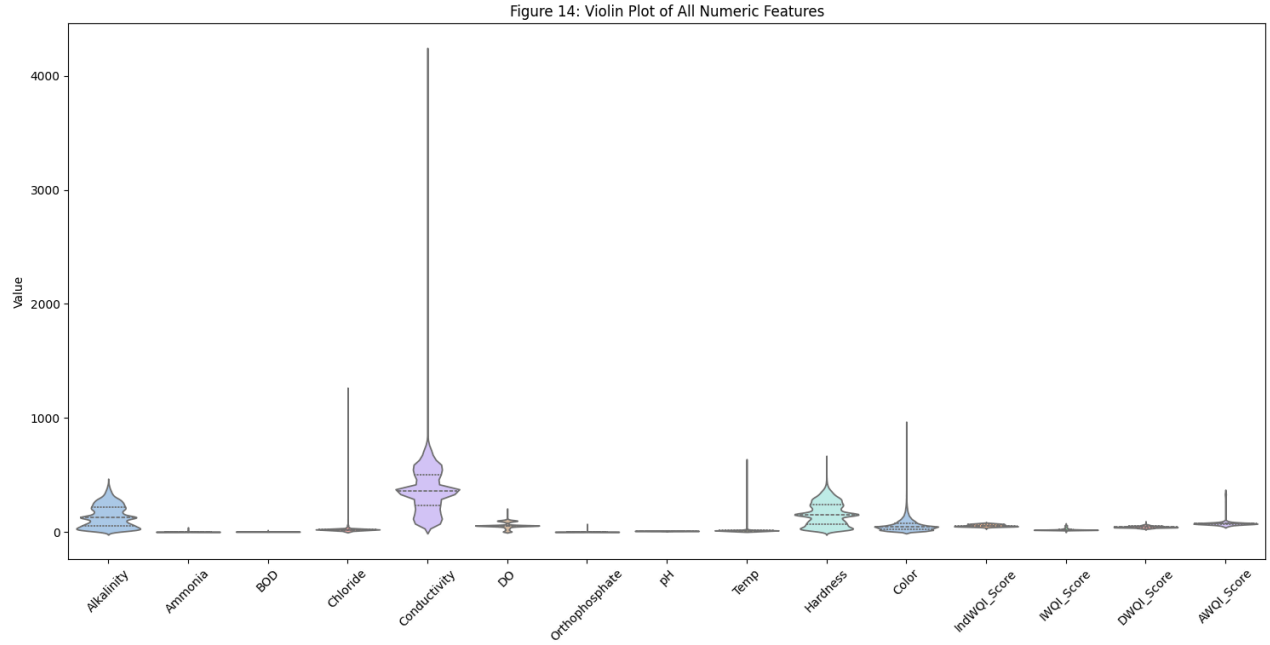
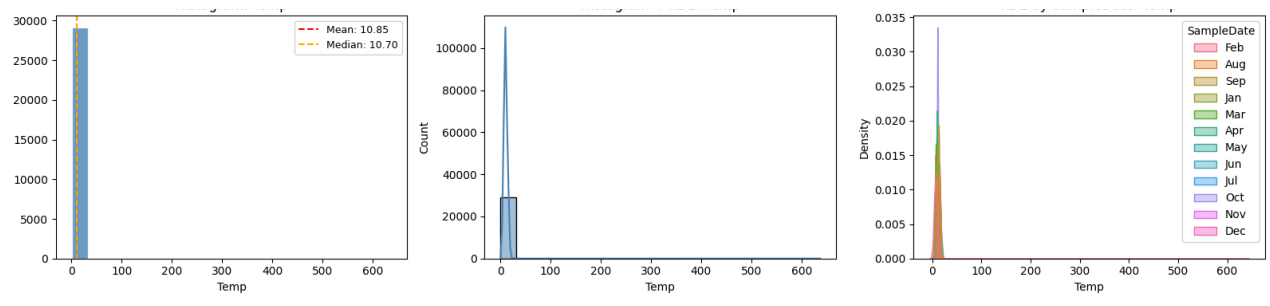
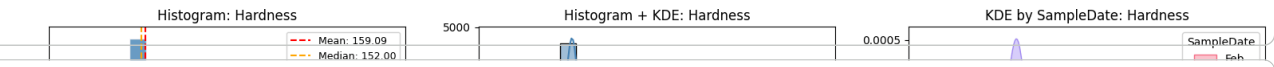


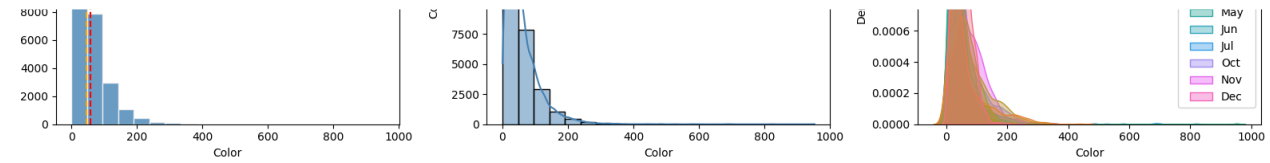
Figure 14: Violin Plot of All Numeric Features

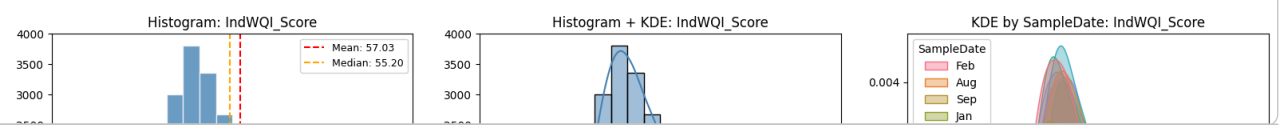







```
# Melt the dataframe so that features become a categorical column
df_melted = df[num_features].melt(var_name='Feature', value_name='Value')

# Plot
plt.figure(figsize=(15, 8))
sns.violinplot(x='Feature', y='Value', data=df_melted, palette='pastel', inner='quartile')
plt.xticks(rotation=45)
plt.title("Figure 14: Violin Plot of All Numeric Features")
plt.xlabel("Features")
plt.ylabel("Value")

plt.ylim(0, 1000)  # Set y-axis max to 2000

plt.tight_layout()
plt.show()
```





```
/tmp/ipykernel_42660/520537645.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable t
  sns.violinplot(x='Feature', y='Value', data=df_melted, palette='pastel', inner='quartile')
```

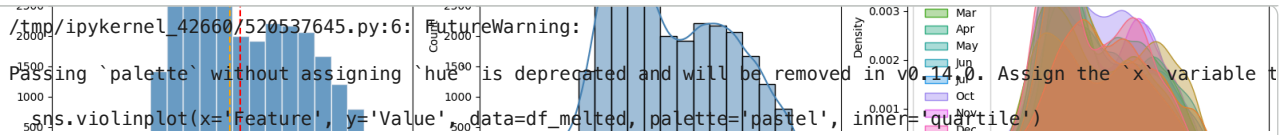
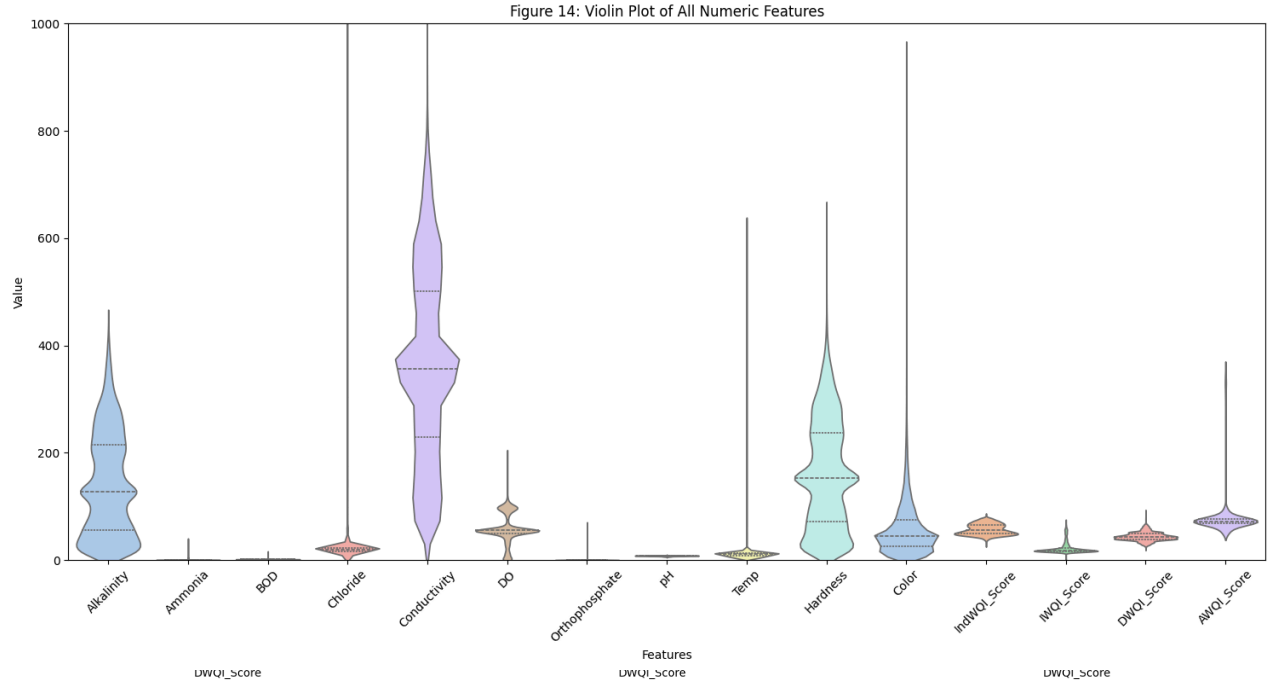
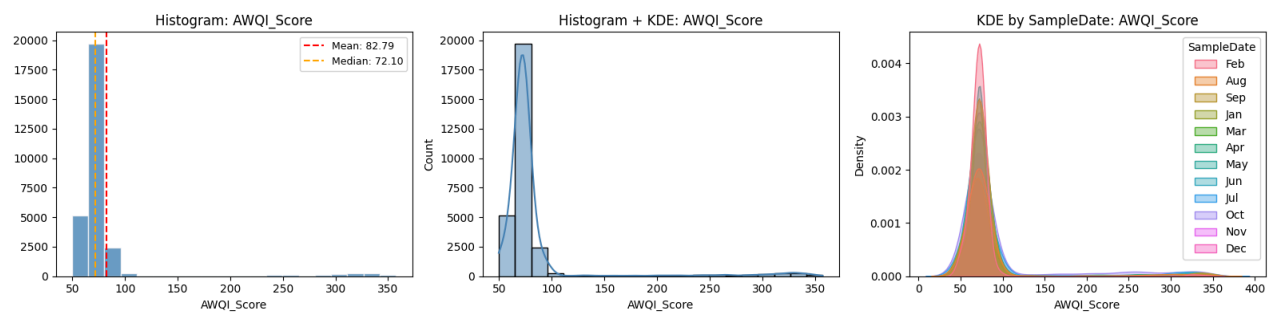


Figure 14: Violin Plot of All Numeric Features



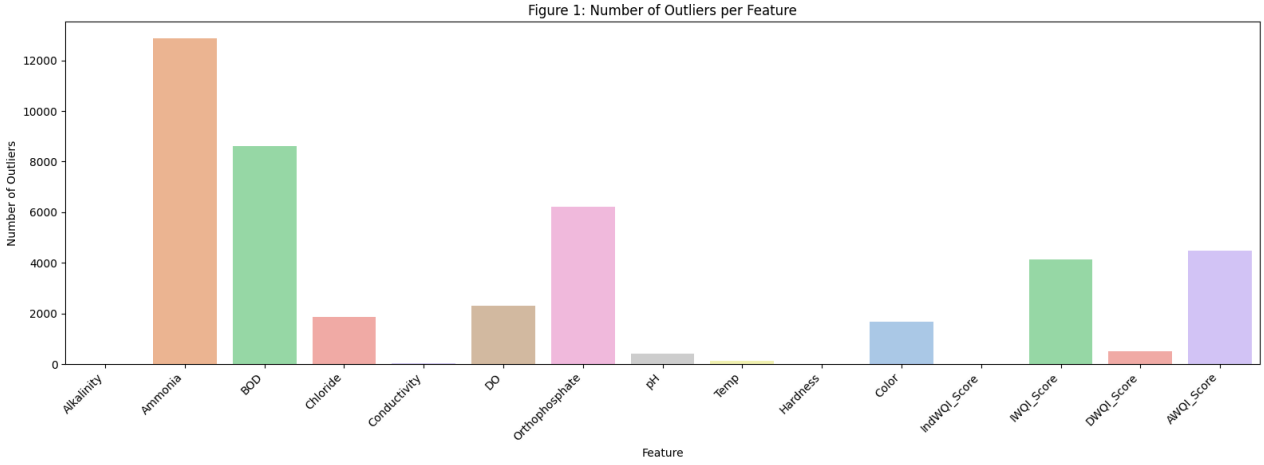



```
from scipy import stats

# Function to calculate stats for a series
def feature_stats(series):
    series = series.dropna()

    # Skewness and kurtosis
    skewness = series.skew()
    kurtosis = series.kurtosis()

    # Normality (Shapiro-Wilk test, sample if large)
    if len(series) >= 3:
        stat, p_value = stats.shapiro(series.sample(n=min(5000, len(series)), random_state=42))
        is_normal = p_value > 0.05
    else:
        is_normal = False

    # Outliers using IQR
    q1, q3 = series.quantile([0.25, 0.75])
    iqr = q3 - q1
    n_outliers = ((series < q1 - 1.5*iqr) | (series > q3 + 1.5*iqr)).sum()

    return pd.Series({
        "Skewness": skewness,
        "Kurtosis": kurtosis,
        "Is_Normal": is_normal,
        "Num_Outliers": n_outliers
    })

# Apply to all numeric features
stats_df = df[num_features].apply(feature_stats).T
stats_df = stats_df.reset_index().rename(columns={"index": "Feature"})
print(stats_df)

# ------------------------------
# Visualization
```

```
# ----------------------------

plt.figure(figsize=(16,6))
sns.barplot(x="Feature", y="Num_Outliers", data=stats_df, palette="pastel")
plt.xticks(rotation=45, ha='right')
plt.title("Figure 1: Number of Outliers per Feature")
plt.ylabel("Number of Outliers")
plt.xlabel("Feature")
plt.tight_layout()
plt.show()

# Skewness plot
plt.figure(figsize=(16,6))
sns.barplot(x="Feature", y="Skewness", data=stats_df, palette="coolwarm")
plt.xticks(rotation=45, ha='right')
plt.title("Figure 2: Skewness of Each Feature")
plt.ylabel("Skewness")
plt.xlabel("Feature")
plt.axhline(0, color='black', linestyle='--')
plt.tight_layout()
plt.show()

# Kurtosis plot
plt.figure(figsize=(16,6))
sns.barplot(x="Feature", y="Kurtosis", data=stats_df, palette="magma")
plt.xticks(rotation=45, ha='right')
plt.title("Figure 3: Kurtosis of Each Feature")
plt.ylabel("Kurtosis")
plt.xlabel("Feature")
plt.axhline(3, color='black', linestyle='--', label="Normal Distribution Kurtosis=3")
plt.legend()
plt.tight_layout()
plt.show()

# Distribution plots (hist + KDE) for all features
for i, feature in enumerate(num_features, 1):
    plt.figure(figsize=(7,4))
    sns.histplot(df[feature], kde=True, color='skyblue', bins=30)
    plt.title(f"Figure {i+3}: Distribution of {feature}")
    plt.xlabel(feature)
    plt.ylabel("Count")
    plt.show()
```

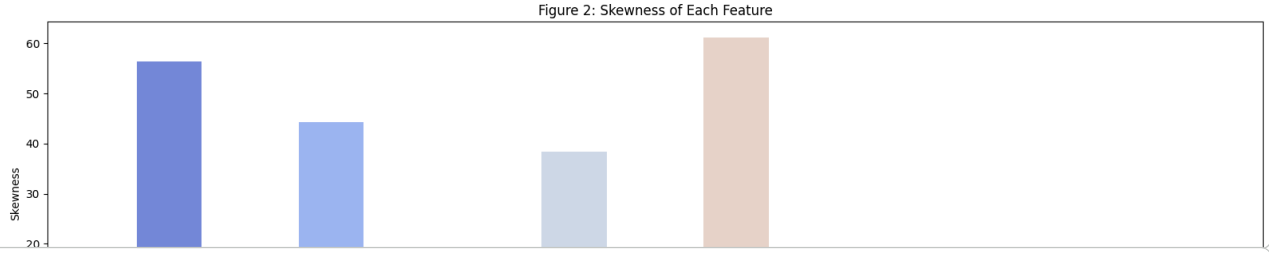
```
            Feature   Skewness     Kurtosis Is_Normal Num_Outliers
0       Alkalinity   0.484602    -0.704908     False            0
1          Ammonia  56.327093  4673.331834     False        12879
2              BOD   8.103517   119.453743     False         8606
3         Chloride  44.313625  2559.327333     False         1861
4     Conductivity   1.297621    20.997343     False           20
5               DO  -0.026754    -0.133513     False         2314
6   Orthophosphate  38.374286  2025.281042     False         6229
7               pH  -0.935719     0.749148     False          406
8             Temp  61.211265  7331.807949     False          134
9         Hardness   0.354629     -0.76224     False            3
10           Color   3.523792    29.994634     False         1685
11    IndWQI_Score   0.306403    -0.910869     False            0
12      IWQI_Score   2.151419     4.189202     False         4141
13      DWQI_Score   0.545878     0.769474     False          506
14      AWQI_Score   4.206465    17.037428     False         4476
/tmp/ipykernel_42660/1679788174.py:40: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable t

  sns.barplot(x="Feature", y="Num_Outliers", data=stats_df, palette="pastel")
```

Figure 1: Number of Outliers per Feature




```
/tmp/ipykernel_42660/1679788174.py:50: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable t

  sns.barplot(x="Feature", y="Skewness", data=stats_df, palette="coolwarm")
```

Figure 2: Skewness of Each Feature




```
# Function to create distribution dashboard
def distribution_dashboard(series, title=""):
    """
    Create 2x2 distribution dashboard and return shape statistics.

    Parameters:
        series (pd.Series): Numeric column
        title (str): Column name for figure title

    Returns:
        dict: {is_normal, skewness, kurtosis, n_outliers}
    """

    # Remove missing values
    data = series.dropna()

    # --------- OUTLIERS (IQR method) ---------
    q1 = data.quantile(0.25)
    q3 = data.quantile(0.75)
    iqr = q3 - q1
    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr
    outliers = data[(data < lower_bound) | (data > upper_bound)]
    n_outliers = len(outliers)

    # --------- NORMALITY TEST (Shapiro) ---------
    if len(data) > 50:
        sample_data = data.sample(50, random_state=42)
    else:
```

```
        sample_data = data
    shapiro_stat, shapiro_p = stats.shapiro(sample_data)
    is_normal = shapiro_p > 0.05

    # --------- SKEWNESS & KURTOSIS ---------
    skewness = round(data.skew(), 4)
    kurtosis = round(data.kurt(), 4)

    # --------- PLOTTING ---------
    plt.figure(figsize=(12, 10))
    if title:
        plt.suptitle(title, fontsize=14)

    # Top-left: Histogram + KDE
    plt.subplot(2, 2, 1)
    sns.histplot(data, kde=True, color='skyblue')
    plt.title("Histogram with KDE")
    plt.xlabel("Value")
    plt.ylabel("Frequency")

    # Top-right: Box plot (horizontal)
    plt.subplot(2, 2, 2)
    sns.boxplot(x=data, color='lightgreen')
    plt.title("Box Plot")
    plt.xlabel("Value")

    # Bottom-left: Violin plot
    plt.subplot(2, 2, 3)
    sns.violinplot(x=data, color='salmon', inner='quartile')
    plt.title("Violin Plot")
    plt.xlabel("Value")

    # Bottom-right: QQ plot
    plt.subplot(2, 2, 4)
    stats.probplot(data, plot=plt)
    plt.title("QQ Plot")

    plt.tight_layout()
    plt.show()

    return {
        "Feature": title,
        "is_normal": is_normal,
        "skewness": skewness,
        "kurtosis": kurtosis,
        "n_outliers": n_outliers
    }

# --------- Generate dashboard for all features ---------
feature_stats_list = []

for feature in num_features:
    stats_dict = distribution_dashboard(df[feature], title=f"Distribution Dashboard: {feature}")
    feature_stats_list.append(stats_dict)

# Convert stats to DataFrame for summary
feature_stats_df = pd.DataFrame(feature_stats_list)
feature_stats_df
```

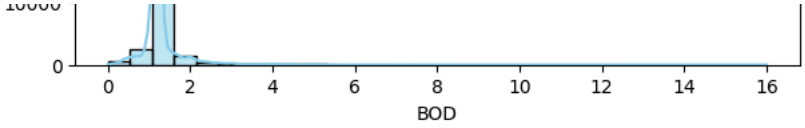
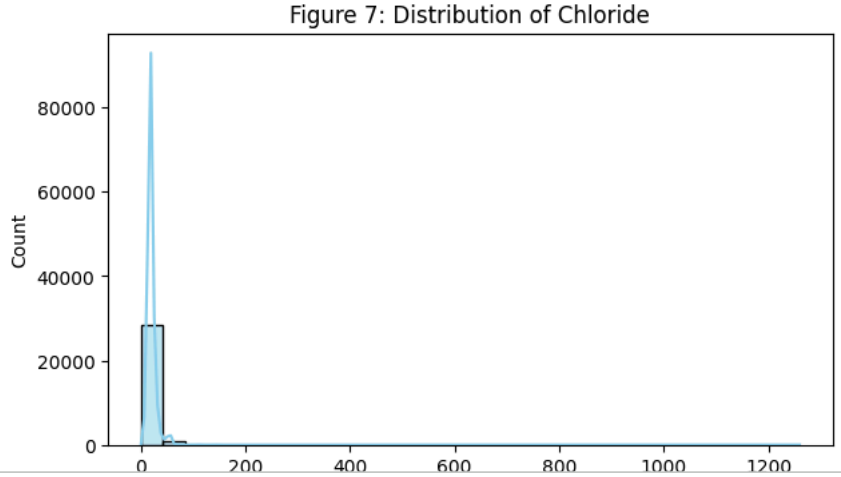
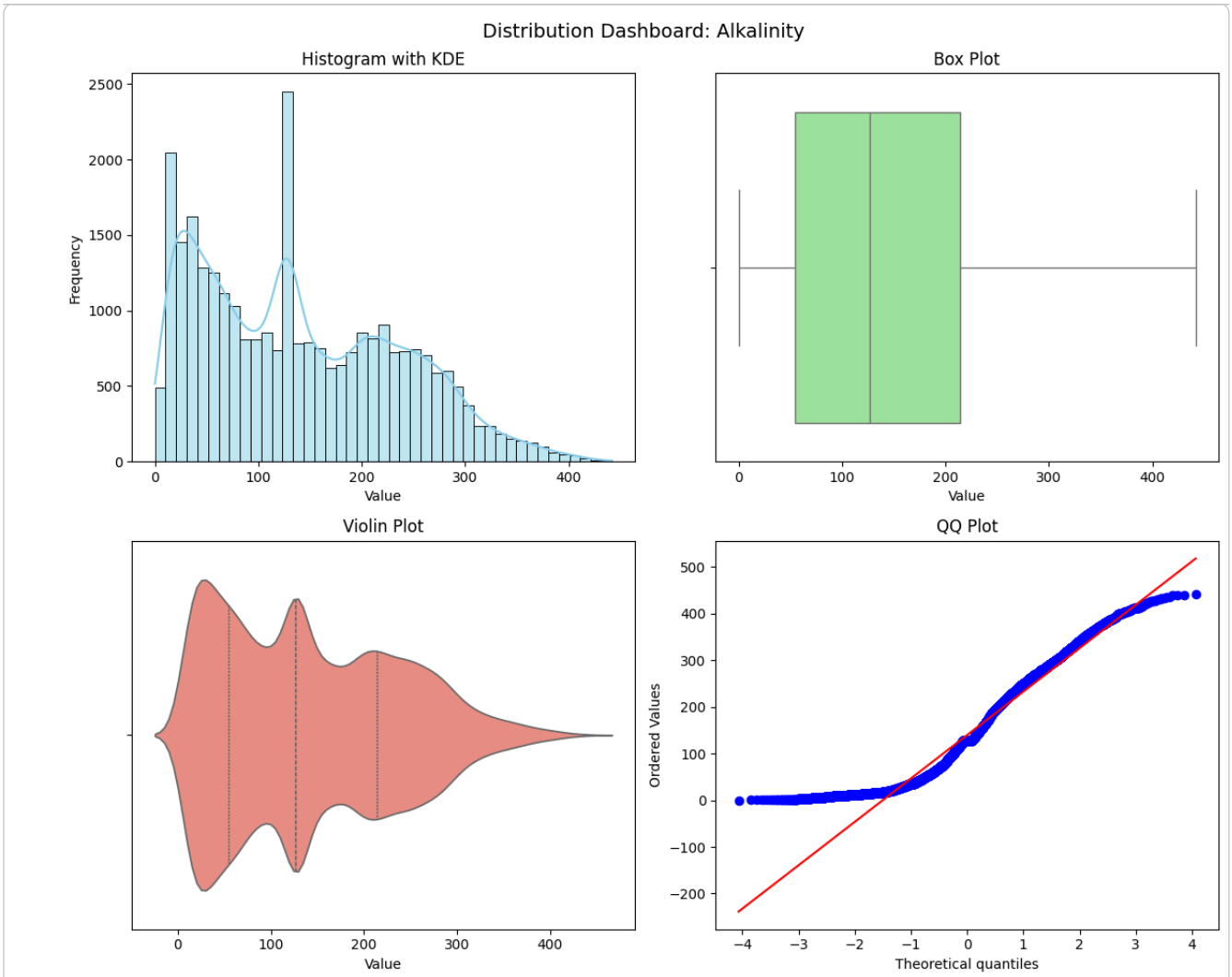
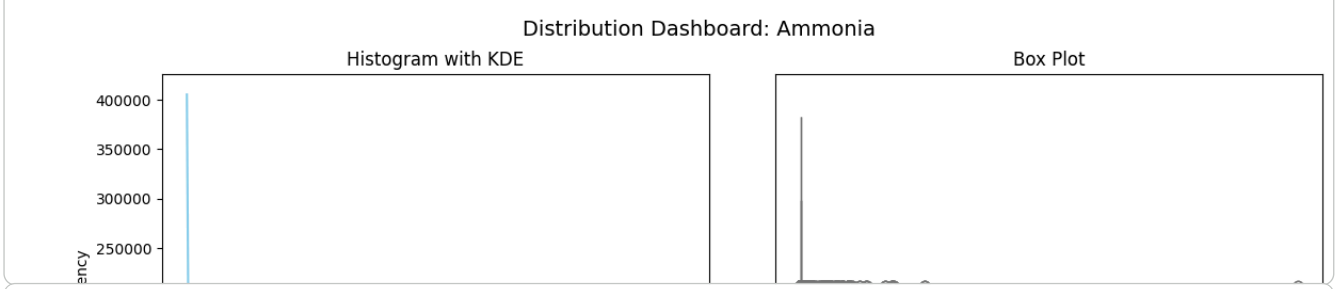


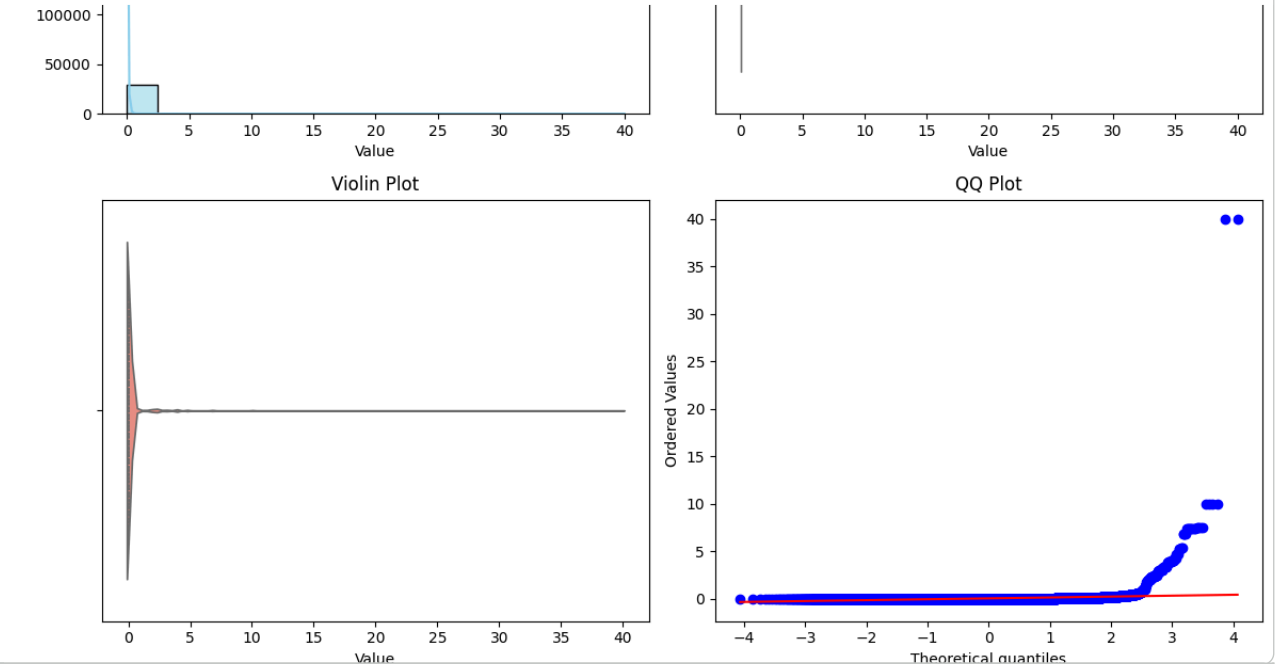
Figure 7: Distribution of Chloride

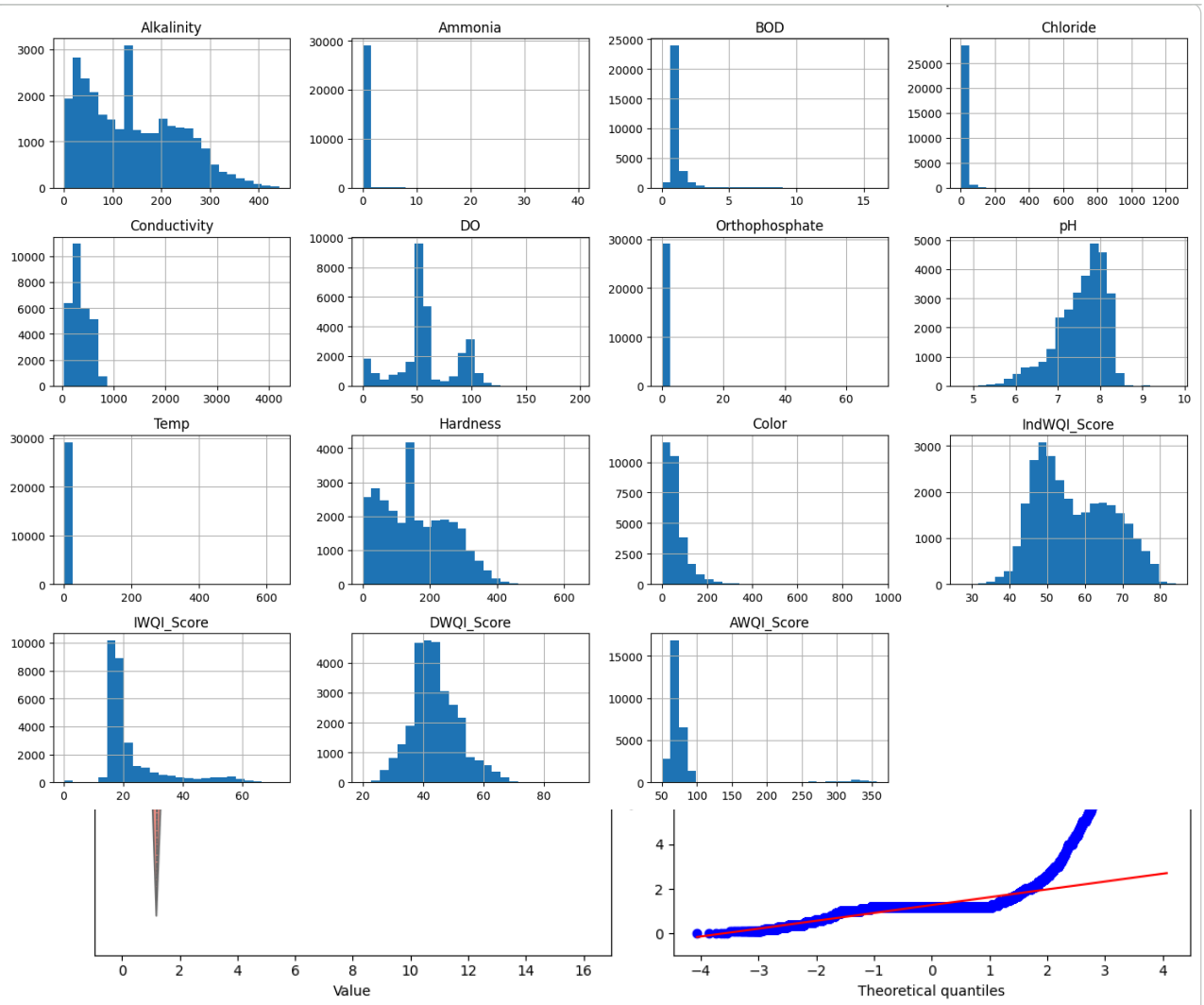


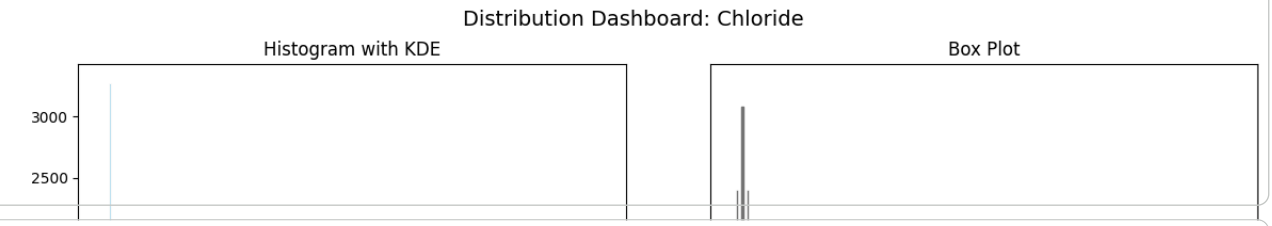




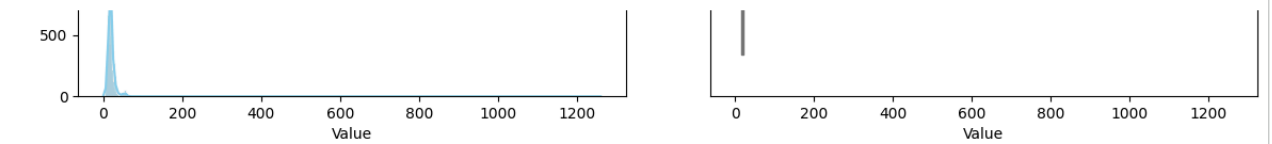

```
df[num_features].hist(figsize=(15, 10), bins=25)
plt.tight_layout()
plt.show()
```

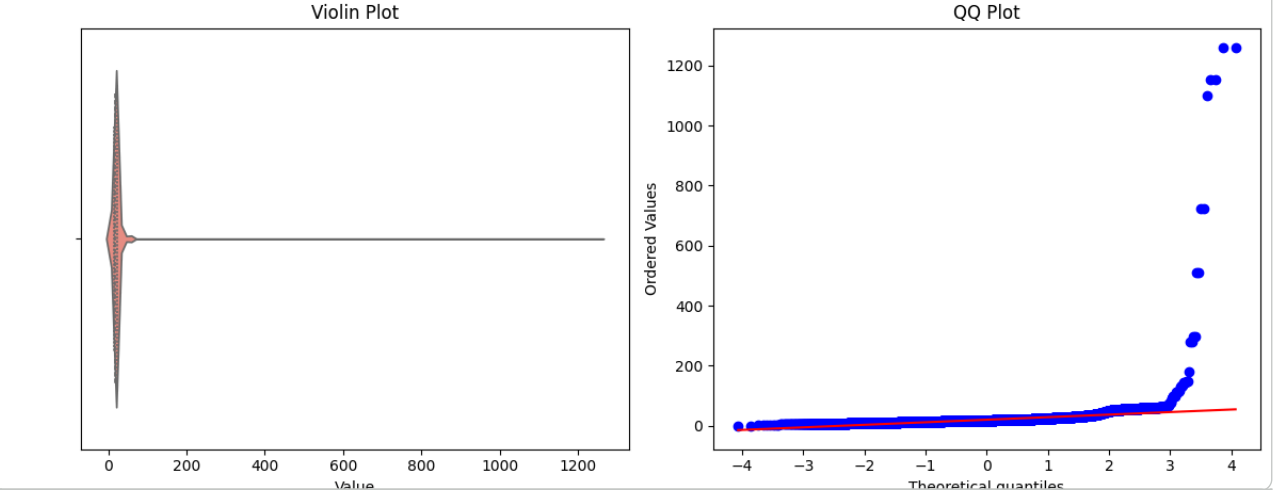


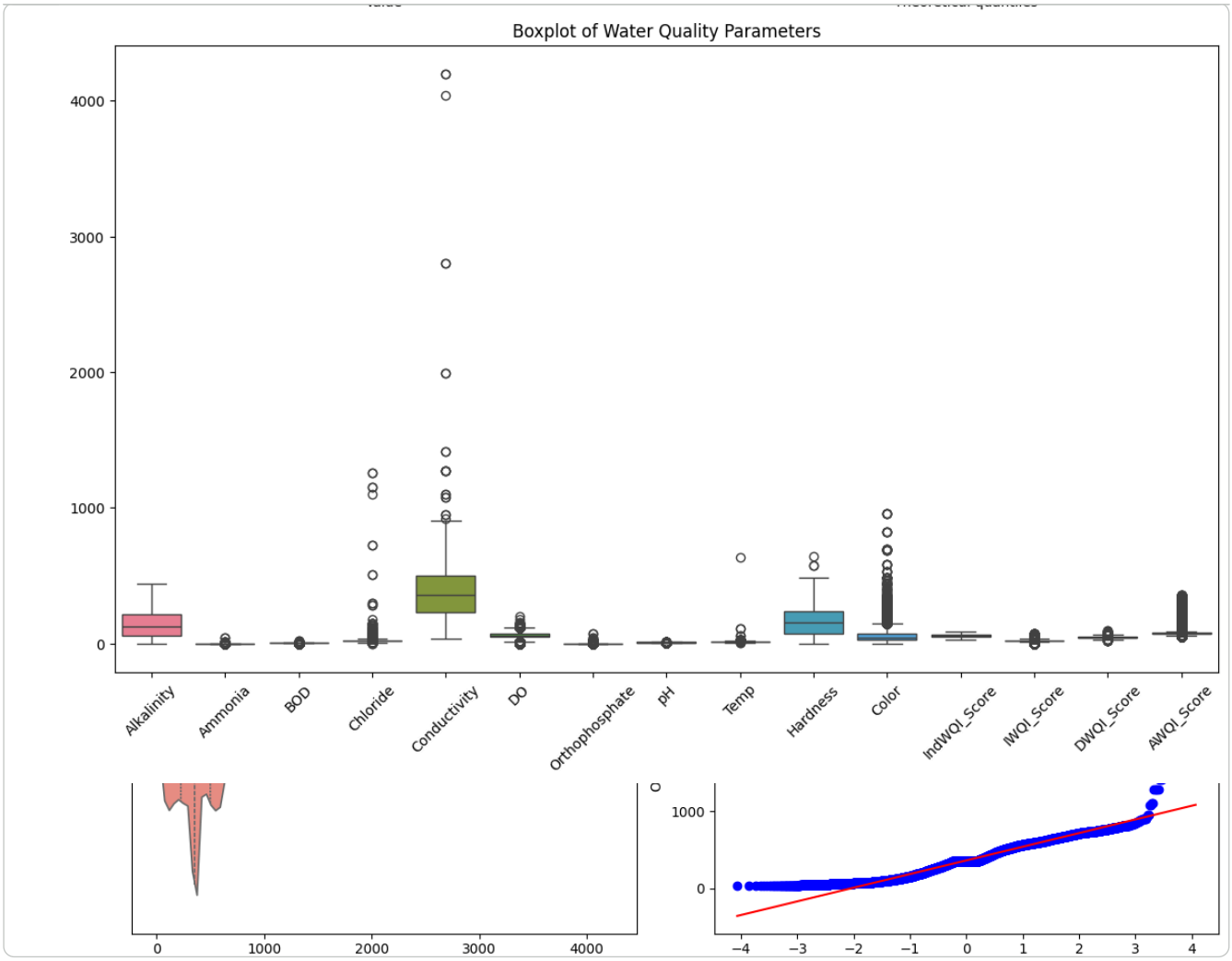




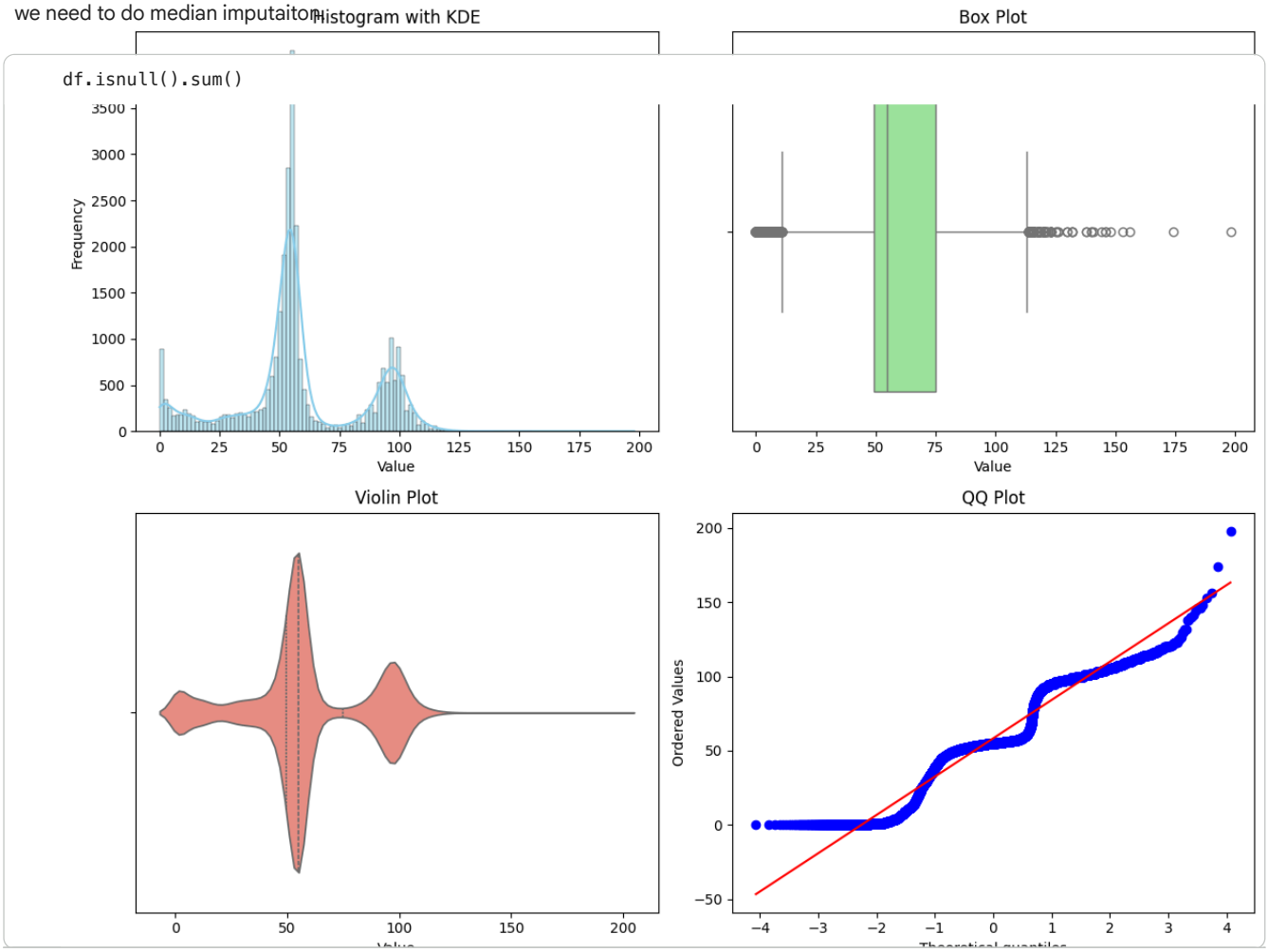

```
# Use boxplots for outlier detection
plt.figure(figsize=(14, 8))
sns.boxplot(data=df[num_features])
plt.xticks(rotation=45)
plt.title("Boxplot of Water Quality Parameters")
plt.show()
```







Water Quality Indices Can not be calculated using NULL values. Therefore, we need to impute it first. As data distribution is not regular, we need to do median imputaiton.




```
df.isnull().sum()
```

```
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipykernel_360/1232201647.py in <cell line: 0>()
     80
     81 for feature in num_features:
---> 82     stats_dict = distribution_dashboard(df[feature], title=f"Distribution Dashboard: {feature}")
     83     feature_stats_list.append(stats_dict)
     84

                                 24 frames
<decorator-gen-2> in __call__(self, obj)

/usr/local/lib/python3.12/dist-packages/matplotlib/backend_bases.py in get_dashes(self)
    762         if self._dashes is None:
    763             return None, None
--> 764         return self._dashes, self._dashoffset
    765
    766     def get_forked(self):
```

WaterbodyName	0
Years	0
SampleDate	0
Alkalinity	0
Ammonia	0
BOD	0
Chloride	0
Conductivity	0
DO	0
Orthophosphate	0
pH	0
Temp	0
Hardness	0
Color	0
IndWQI	0
IndWQI_Score	0
IWQI	0
IWQI_Score	0
DWQI	0
DWQI_Score	0
AWQI	0
AWQI_Score	0

dtype: int64

1. Missing Value Analysis:

Median Imputatation Done Before Calculating WQI's

```
# # Apply median imputation
# for col in num_features:
#     df[col].fillna(df[col].median(), inplace=True)
```

2. Data Type Conversion::

All water quality parameters found from the data source was in unified data type. Therefore we did not had to make any explicit conversions. As we will work on regressive model, any encoding is not required still.

3. Outlier Detection and Treatment:

IQR Method for Outlier Detection

```
outliers_iqr = {}

for col in num_features:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)][col]
    outliers_iqr[col] = len(outliers)

print("Number of outliers detected via IQR method:")
for col, count in outliers_iqr.items():
    print(f"{col}: {count}")
```

```
Number of outliers detected via IQR method:
Alkalinity: 0
Ammonia: 12879
BOD: 8606
Chloride: 1861
Conductivity: 20
DO: 2314
Orthophosphate: 6229
pH: 406
Temp: 134
Hardness: 3
Color: 1685
IndWQI_Score: 0
IWQI_Score: 4141
DWQI_Score: 506
AWQI_Score: 4476
```

Z-score Method for Outlier Detection

Z-Score is preferable as we need to exclude less outliers. All WQI's are eaqual important, whether its outlier or not.

```
from scipy import stats

outliers_z = {}

for col in num_features:
    z_scores = np.abs(stats.zscore(df[col].fillna(df[col].median())))
    outliers = np.where(z_scores > 3)[0]  # threshold: 3 std dev
    outliers_z[col] = len(outliers)

print("Number of outliers detected via Z-score method:")
for col, count in outliers_z.items():
    print(f"{col}: {count}")
```

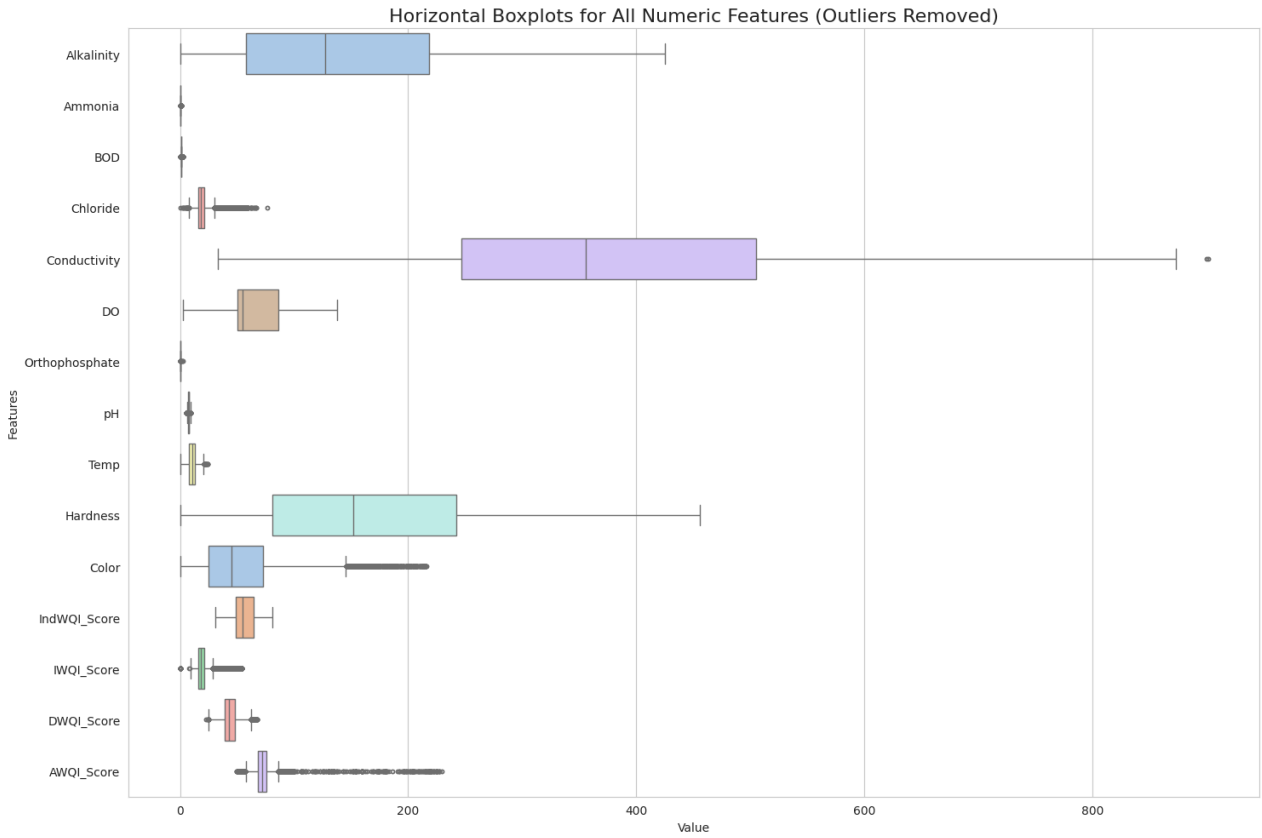
```
Number of outliers detected via Z-score method:
Alkalinity: 16
Ammonia: 150
BOD: 487
Chloride: 39
Conductivity: 20
DO: 11
Orthophosphate: 89
pH: 239
Temp: 6
Hardness: 16
Color: 477
IndWQI_Score: 0
IWQI_Score: 1010
DWQI_Score: 112
AWQI_Score: 1194
```

Removing outliers using Z-Score for All numeric values

```
from scipy import stats

# Compute Z-scores for all numerical features
z_scores = np.abs(stats.zscore(df[num_features]))
threshold = 3

# Keeping rows where all feature Z-scores are below the threshold
df_no_outliers = df[(z_scores < threshold).all(axis=1)]

print(f"Original dataset shape: {df.shape}")
print(f"Dataset shape after removing outliers: {df_no_outliers.shape}")
```

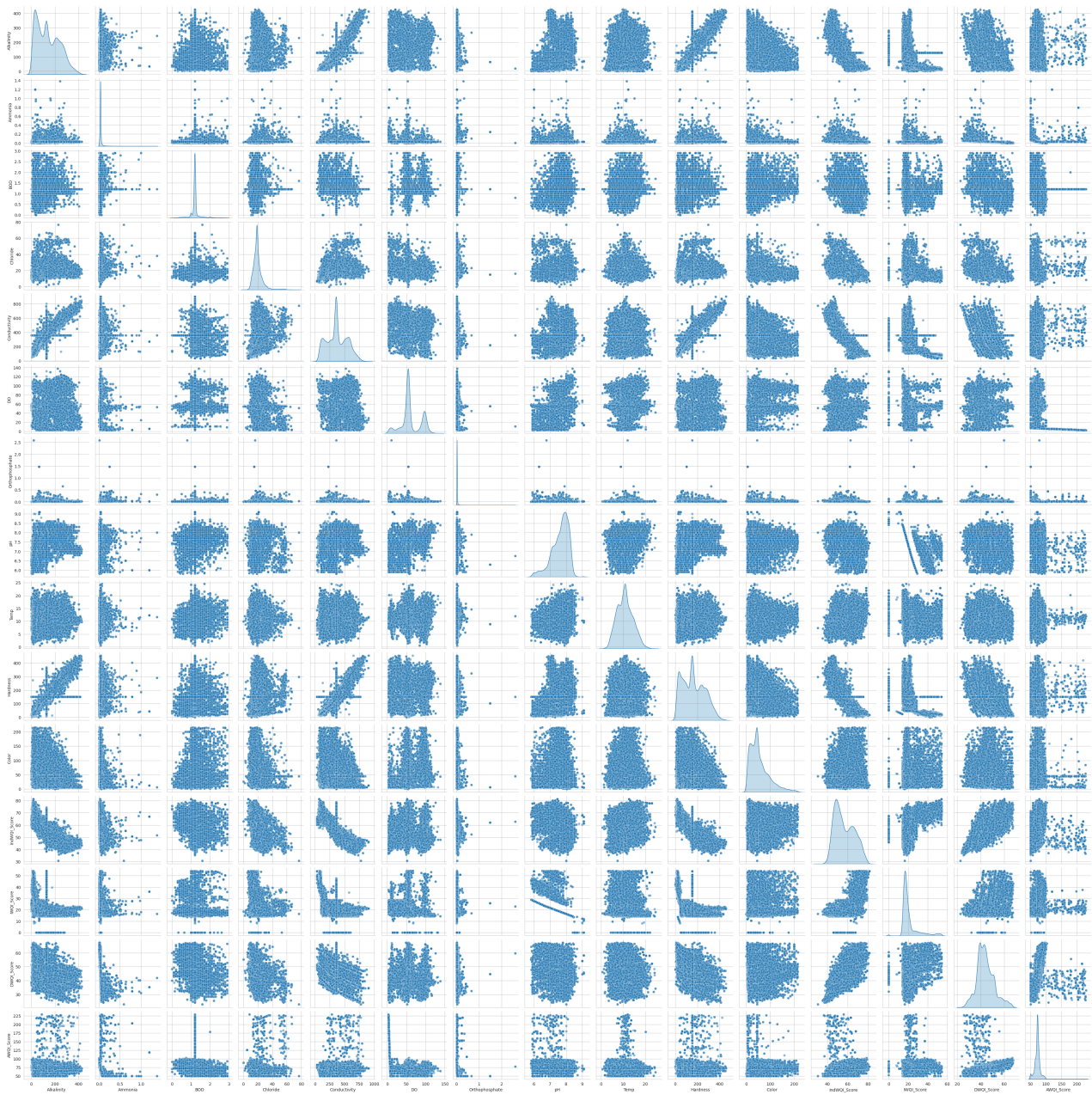
```
Original dataset shape: (29159, 22)
Dataset shape after removing outliers: (25633, 22)
```

```
plt.figure(figsize=(15, 10))
sns.boxplot(data=df_no_outliers[num_features],
            orient="h",
            palette="pastel",
            fliersize=3)
plt.title("Horizontal Boxplots for All Numeric Features (Outliers Removed)", fontsize=16)
plt.xlabel("Value")
plt.ylabel("Features")
plt.tight_layout()
plt.show()
```




```
# Pairplot with KDE on diagonals and alpha for transparency
sns.pairplot(df_no_outliers[num_features], diag_kind='kde', plot_kws={'alpha':0.5})
plt.suptitle("Figure 13: Pairwise Relationships (First 6 Numeric Features, Outliers Removed)", y=1.02)
plt.show()
```

Figure 13: Pairwise Relationships (First 6 Numeric Features, Outliers Removed)




```
# Split features into two groups for pairplot to avoid freezing
feature_groups = [num_features[:6], num_features[6:]]

for i, group in enumerate(feature_groups, 1):
```

```
    sns.pairplot(df_no_outliers[group], diag_kind='kde', plot_kws={'alpha':0.5})
    plt.suptitle(f"Figure 14.{i}: Pairwise Relationships (Features Group {i}, Outliers Removed)", y=1.02)
    plt.show()
```

Figure 14.1: Pairwise Relationships (Features Group 1, Outliers Removed)

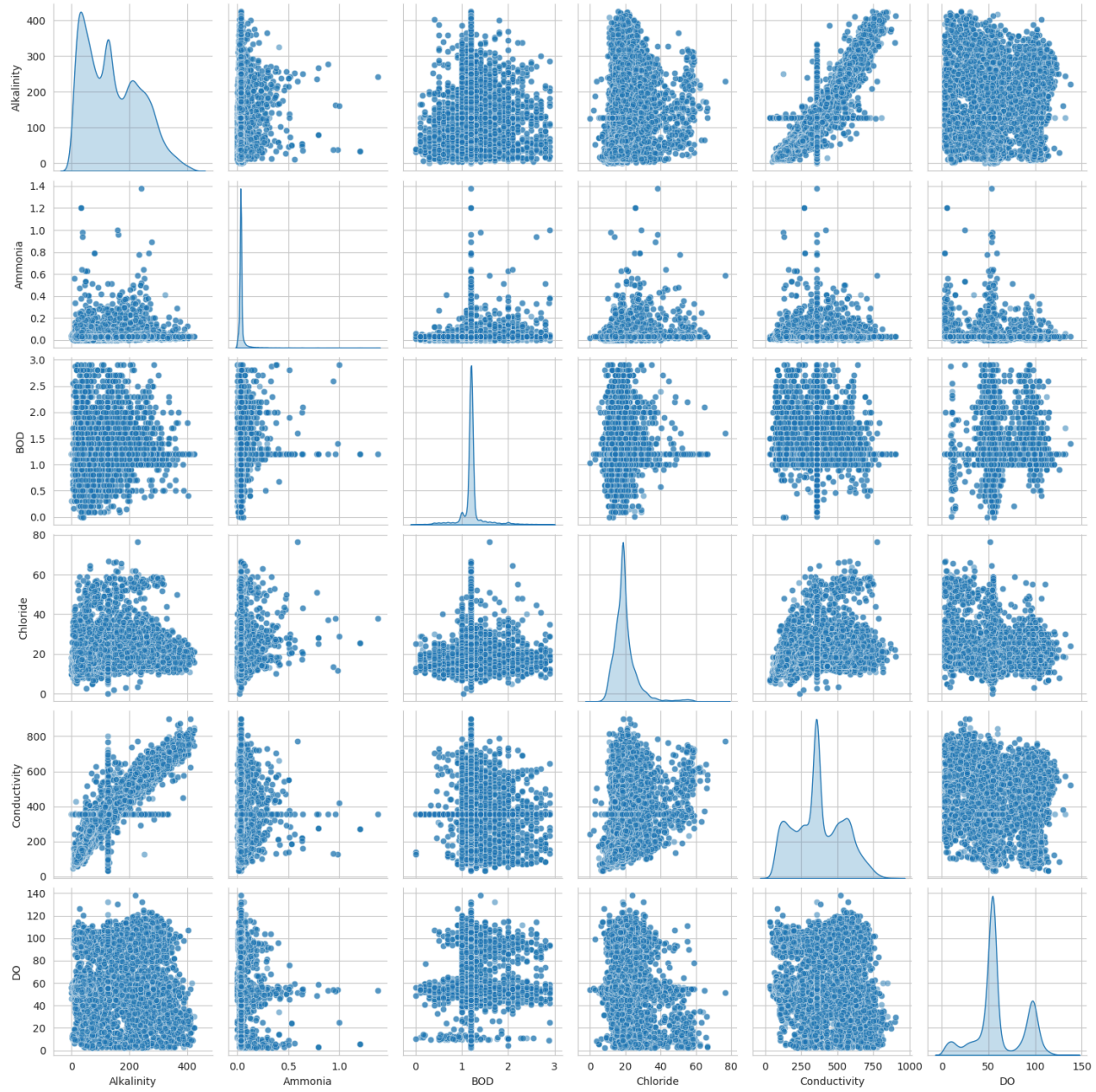
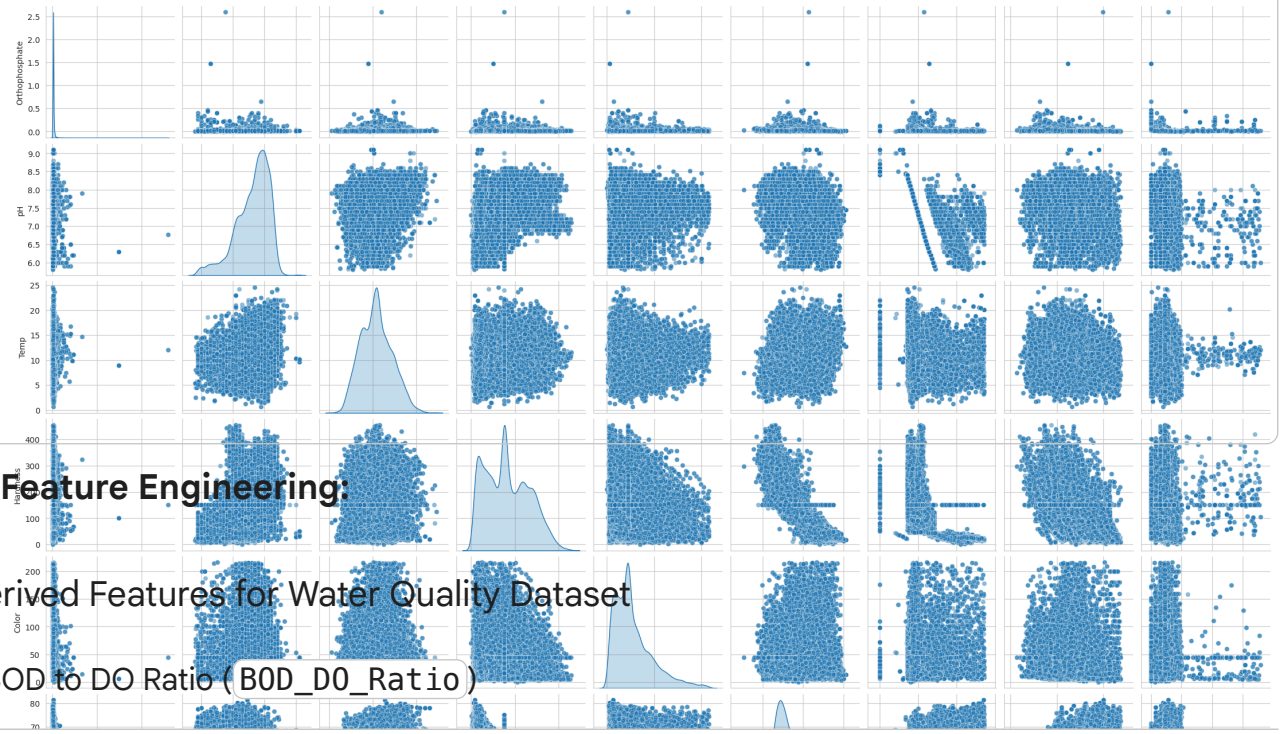


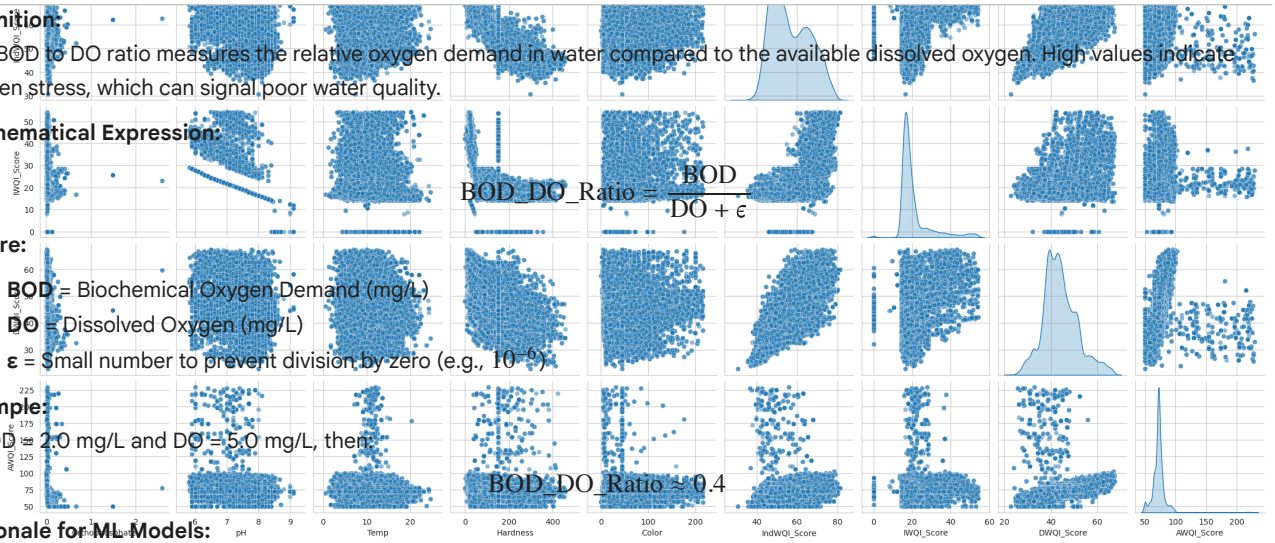
Figure 14.2: Pairwise Relationships (Features Group 2, Outliers Removed)



4. Feature Engineering:

Derived Features for Water Quality Dataset

i. BOD to DO Ratio (`BOD_DO_Ratio`)



Definition:

The BOD to DO ratio measures the relative oxygen demand in water compared to the available dissolved oxygen. High values indicate oxygen stress, which can signal poor water quality.

Mathematical Expression:

$$\text{BOD\_DO\_Ratio} = \frac{\text{BOD}}{\text{DO} + \epsilon}$$

Where:

- **BOD** = Biochemical Oxygen Demand (mg/L)
- **DO** = Dissolved Oxygen (mg/L)
- **ε** = Small number to prevent division by zero (e.g., 10^{-6})

Example:

If BOD = 2.0 mg/L and DO = 5.0 mg/L, then:

$$\text{BOD\_DO\_Ratio} \approx 0.4$$

Rationale for ML Models:

- Combines two related water quality indicators into a single feature.
- Can improve model sensitivity to oxygen-related water quality issues, enhancing WQI prediction.

---

ii. Hardness to Alkalinity Ratio (`Hardness_Alkalinity_Ratio`)

Definition:

This ratio quantifies the relative contribution of water hardness to alkalinity. It reflects water scaling potential and buffering capacity.

Mathematical Expression:

$$\text{Hardness\_Alkalinity\_Ratio} = \frac{\text{Hardness}}{\text{Alkalinity} + \epsilon}$$

Where:

- **Hardness** = Total water hardness (mg/L as $CaCO_3$)
- **Alkalinity** = Water alkalinity (mg/L as $CaCO_3$)
- **ε** = Small number to prevent division by zero

Example:

If Hardness = 120 mg/L and Alkalinity = 100 mg/L, then:

$$\text{Hardness\_Alkalinity\_Ratio} \approx 1.2$$

Rationale for ML Models:

- Captures interaction between two correlated chemistry features.
- Useful for detecting water scaling or buffering patterns that affect overall water quality.

Derived Features:

Feature 1: BOD to DO Ratio (BOD_DO_Ratio)

Rationale:

BOD (Biochemical Oxygen Demand) indicates the amount of oxygen consumed by microorganisms. DO (Dissolved Oxygen) measures available oxygen in water. High BOD combined with low DO often signals poor water quality. A BOD/DO ratio can serve as a single combined feature to capture oxygen stress in the water, which may improve WQI prediction or classification.

```
# Feature 1: BOD to DO ratio
df_no_outliers['BOD_DO_Ratio'] = df_no_outliers['BOD'] / (df_no_outliers['DO'] + 1e-6)  # small epsilon to avoid

# Inspect
df_no_outliers[['BOD', 'DO', 'BOD_DO_Ratio']].head()
```